

BỔ SUNG KIẾN THỨC GITHUB

Hoàn thiện Git/GitHub cho dự án thực tế

Pull Request • Code Review • Conflict • CI/CD • Release • Security • Bài tập

Mục tiêu: File này là phần bổ sung cho tài liệu branch trước đó. Mục tiêu là giúp bạn không chỉ biết tạo branch, mà còn hiểu cách dùng GitHub như một người làm dự án thật: quản lý task, review, bảo vệ nhánh chính, tự động test/build và release.

Cách học tài liệu này

- Đọc theo thứ tự từ Module 1 đến Module 16, vì các phần phía sau dùng lại khái niệm phía trước.
- Mỗi khi gặp lệnh Git, hãy tự tạo repo test để chạy thử, không thực hành trực tiếp trên dự án quan trọng.
- Đừng học thuộc toàn bộ lệnh. Hãy nhớ tình huống: muốn lưu, muốn kéo, muốn sửa sai, muốn review, muốn release.
- Khi làm team, ưu tiên an toàn: tạo Pull Request, chạy test, review, rồi mới merge.

Bản đồ kiến thức GitHub cần hoàn thiện

Nhóm kiến thức	Bạn cần hiểu được gì?	Dùng khi nào?
Git nền tảng	Working directory, staging, local repo, remote repo, commit history.	Mọi ngày khi code.
Pull Request	Đề xuất gộp code, review, chạy check, merge.	Khi làm team hoặc muốn bảo vệ branch chính.
Conflict	Tại sao xảy ra, đọc marker, sửa, tiếp tục merge/rebase.	Khi nhiều người sửa cùng vùng code.
Review	Kiểm tra logic, style, bảo mật, test, khả năng maintain.	Trước khi merge code.
Issue/Project	Quản lý task, bug, feature theo board.	Khi dự án có nhiều việc và nhiều người.
CI/CD	Tự động build, test, deploy khi push/PR.	Dự án thật, cần kiểm tra tự động.
Release/Security	Tag, version, branch protection, secrets, .gitignore.	Khi đưa sản phẩm ra môi trường thật.

Module 1— Git nền tảng trước khi đi sâu vào GitHub

GitHub là nơi lưu repo từ xa, nhưng phần lõi vẫn là Git. Nếu chưa hiểu Git lưu code theo từng lớp, bạn sẽ rất dễ nhầm giữa add, commit, push và merge.

1.1. Bốn vùng quan trọng trong Git

WorkingDirectory ->StagingArea ->Local Repository ->Remote Repository

File bạn đang sửa ->File đã chọn để commit ->Commit ở máy bạn ->GitHub

Vùng	Giải thích dễ hiểu	Lệnh liên quan
WorkingDirectory	Thư mục code bạn đang mở trên máy. File sửa ở đây chưa chắc đã được Git lưu.	git status git diff git restore
StagingArea	Khu vực chuẩn bị commit. Bạn chọn file nào thì file đó sẽ vào commit tiếp theo.	git add. git reset <file>
LocalRepository	Kho Git trên máy bạn. Commit tạo ra ở đây.	git commit git log git show
Remote Repository	Repo trên GitHub. Push mới đưa commit từ máy lên đây.	git push git pull git fetch

1.2. Bộ lệnh nền tảng phải chắc

```

# Kiểm tra trạng thái hiện tại
git status

# Xem file đã thay đổi gì so với lần commit gần nhất
git diff

# Đưa toàn bộ thay đổi vào staging
git add.

# Lưu một mốc thay đổi ở máy mình
git commit -m "feat: add user login API"

# Đẩy commit lên GitHub
git push

# Kéo code mới nhất từ GitHub về branch hiện tại
git pull

# Cập nhật thông tin remote nhưng chưa gộp vào code local
git fetch origin

# Xem lịch sử commit ngắn gọn
git log --oneline --graph --all
    
```

Lưu ý: git pull thực chất thường tương đương fetch + merge. Vì vậy khi chỉ muốn xem GitHub có branch/commit mới nào, dùng git fetch an toàn hơn.

1.3. Bài tập nhỏ

- Tạo repo git-practice trên GitHub.
- Clone repo về máy.
- Tạo file README.md, commit, push.
- Sửa README.md lần 2, chạy git diff trước khi commit.
- Dùng git log --oneline để xem lịch sử.

Module 2 — Commit message và cách commit chuyên nghiệp

Commit không chỉ là lưu code. Commit là lịch sử kể lại dự án đã thay đổi như thế nào. Dự án càng lớn thì commit càng cần rõ ràng.

2.1. Commit xấu và commit tốt

Không nên	Nên
git commit -m "update"	git commit -m "feat: add user login API"
git commit -m "fix"	git commit -m "fix: correct cart total calculation"
git commit -m "abc"	git commit -m "docs: update local setup guide"
Một commit chứa login + payment + sửa README	Mỗi commit chỉ nên tập trung vào một ý nghĩa chính

2.2. Conventional Commits

```

feat: add product CRUD APIs
fix: correct order status transition
docs: update API usage guide
refactor: split order service from checkout service
chore: configure Docker Compose
test: add unit tests for CartService
ci: add GitHub Actions build workflow
style: format controller code
    
```

Prefix	Ý nghĩa	Ví dụ
feat	Thêm chức năng mới.	feat: add payment callback endpoint
fix	Sửa lỗi.	fix: prevent duplicate order creation
docs	Sửa tài liệu.	docs: add README setup instructions
refactor	Cải tổ code, không đổi hành vi bên ngoài.	refactor: extract PriceCalculator
test	Thêm/sửa test.	test: add login validation tests
chore	Cấu hình, dependency, việc phụ.	chore: update Maven dependencies
ci	Pipeline CI/CD.	ci: run tests on pull request

2.3. Atomic commit

Atomic commit nghĩa là một commit nên đại diện cho một thay đổi có ý nghĩa, càng ít lần việc càng tốt.

```

# Không tốt: một commit làm quá nhiều việc
git commit -m "feat: add login, cart, payment, update docs"
    
```



```
# Tốt hơn
git commit -m "feat: add user login API"
git commit -m "feat: add cart item API"
git commit -m "docs: update API examples"
```

Lưu ý: Commit nhỏ và rõ giúp review dễ hơn, revert dễ hơn, tìm lỗi dễ hơn.

Module 3 — Pull Request: cửa kiểm soát trước khi merge

Pull Request, thường gọi là PR, là yêu cầu gộp code từ branch này sang branch khác. PR giúp team kiểm tra code trước khi đưa vào branch chính.

3.1. Luồng Pull Request chuẩn

```
develop -> tạo feature branch -> code -> commit -> push -> tạo Pull Request -> review -> CI pass -> merge -> delete branch
```

10. Tạo branch từ develop hoặc main tùy workflow.
11. Code và commit trên branch riêng.
12. Push branch lên GitHub.
13. Tạo Pull Request từ feature/... vào develop hoặc main.
14. Gắn reviewer, mô tả thay đổi, link issue nếu có.
15. Chờ review và check tự động pass.
16. Merge và xóa branch sau khi xong.

3.2. Lệnh trước khi tạo PR

```
# Đảm bảo đang ở đúng branch feature
git switch feature/user-login

# Cập nhật code mới nhất từ branch chính
git fetch origin
git merge origin/develop
# hoặc: git rebase origin/develop nếu team thống nhất dùng rebase

# Chạy test local trước khi push
./mvnw test
# hoặc: npm test

# Push branch lên GitHub
git push -u origin feature/user-login
```

3.3. Mẫu mô tả Pull Request

```
## Mục tiêu
Thêm API đăng nhập cho user.

## Thay đổi chính
- Tạo endpoint POST /api/auth/login
- Validate email và password
- Trả về JWT token khi đăng nhập thành công

## Cách test
1. Chạy app local
2. Gọi POST /api/auth/login với email/password hợp lệ
```

```

3. Kiểm tra response có token

## Ảnh hưởng
- Không thay đổi database schema
- Không ảnh hưởng API register

## Liên kết issue
Closes #12
    
```

3.4. Draft Pull Request

Draft PR dùng khi code chưa xong nhưng bạn muốn team thấy tiến độ, góp ý sớm hoặc kiểm tra hướng làm.

Lưu ý: Draft PR không nên merge. Khi code đã sẵn sàng, chuyển từ Draft sang Ready for review.

3.5. Checklist trước khi mở PR

- Branch được tạo từ đúng branch gốc.
- Tên branch rõ ràng: feature/user-login, bugfix/cart-total.
- Commit message rõ nghĩa.
- Không có file rác: .env, target/, node_modules/, log.
- Đã chạy test hoặc ít nhất chạy được project local.
- PR có mô tả thay đổi và cách test.
- Không tự merge khi chưa được review nếu làm team.

Module 4 — Code Review: xem code như người làm dự án thật

Review code không phải để bắt lỗi vặt. Mục tiêu là bảo vệ chất lượng dự án, giúp code dễ hiểu, dễ sửa, ít bug và an toàn hơn.

4.1. Review cần xem những gì?

Nhóm kiểm tra	Câu hỏi cần hỏi
Logic nghiệp vụ	Code có đúng yêu cầu không? Có bỏ sót case lỗi không?
Thiết kế code	Controller có quá nhiều logic không? Service có rõ trách nhiệm không?
Đọc hiểu	Tên biến, tên hàm, tên class có rõ nghĩa không?
Bảo mật	Có lộ password/API key không? Có validate input không?
Database	Query có nguy hiểm không? Có thiếu transaction không?
Test	Có test cho case chính và case lỗi không?
Ảnh hưởng cũ	Có phá API hoặc chức năng đang chạy không?

4.2. Ví dụ comment review tốt

```

Nên tách logic tính totalPrice ra CartService để controller gọn hơn.

Chỗ này cần xử lý trường hợp productId không tồn tại, hiện tại có thể gây NullPointerException.

Tên biến hơi khó hiểu. Có thể đổi thành product hoặc productEntity.

API này nhận email từ client, nên validate định dạng email trước khi query database.
    
```


Nên thêm test cho trường hợp password sai.

4.3. Cách phản hồi review

Reviewer: Chỗ này nên validate email trước khi xử lý.

Author: Đã thêm validate email trong AuthService và bổ sung test case invalid email.

Reviewer: Hàm này hơi dài, nên tách bớt.

Author: Đã tách validateOrderItems() và calculateTotalPrice() để dễ đọc hơn.

Lưu ý: Khi bị review, đừng xem đó là bị chê. Review là cơ chế bảo vệ codebase chung.

Module 5 — Merge conflict: hiểu và xử lý đúng

Conflict xảy ra khi Git không tự quyết định được nên lấy thay đổi nào, thường vì nhiều người sửa cùng một vùng code.

5.1. Ví dụ conflict

```
<<<<<<HEAD
return "Login success";
=====
return "User login successfully";
>>>>>>feature/login-message
```

Ý nghĩa:

- Phần giữa <<<<<<HEAD và ===== là code hiện tại ở branch bạn đang đứng.
- Phần giữa ===== và >>>>>> là code từ branch đang được merge/rebase vào.
- Bạn phải sửa thủ công để còn lại code cuối cùng mong muốn.

5.2. Quy trình xử lý conflict khi merge

```
git switch feature/order
git fetch origin
git merge origin/develop

# Nếu có conflict:
git status
# Mở file bị conflict, sửa thủ công marker <<<<<<=====>>>>>>

# Sau khi sửa xong:
git add.
git commit

# Push lại branch nếu cần
git push
```

5.3. Quy trình xử lý conflict khi rebase

```
git switch feature/order
git fetch origin
git rebase origin/develop

# Nếu có conflict:
git status
# Sửa file conflict
```

```
git add.
git rebase --continue

# Nếu còn conflict ở commit tiếp theo, tiếp tục sửa và rebase --continue

# Sau rebase, nếu branch đã từng push, có thể cần:
git push --force-with-lease
```

Cảnh báo: Chỉ dùng `--force-with-lease` khi bạn hiểu mình đang làm gì. Không dùng `git push --force` tùy tiện trên branch chung nhiều người cùng code.

5.4. Khi muốn hủy merge/rebase

```
# Hủy merge đang dở
git merge --abort

# Hủy rebase đang dở
git rebase --abort
```

Module 6 — Merge và Rebase

Merge và rebase đều dùng để cập nhật hoặc gộp lịch sử, nhưng cách tạo lịch sử khác nhau.

Tiêu chí	Merge	Rebase
Ý tưởng	Gộp nhánh và giữ lịch sử phân nhánh.	Đặt commit của bạn lên trên lịch sử mới nhất.
Lịch sử	Có merge commit, nhìn đúng quá trình làm việc.	Lịch sử thẳng và gọn hơn.
Dễ cho người mới	Dễ hiểu hơn.	Cần cẩn thận hơn.
Dùng khi	Gộp PR, cập nhật branch chung.	Làm sạch lịch sử branch cá nhân trước PR.
Cảnh báo	Có thể nhiều merge commit.	Không rebase branch mà nhiều người đang cùng dùng nếu chưa thống nhất.

6.1. Cập nhật branch feature bằng merge

```
git switch feature/payment
git fetch origin
git merge origin/develop
git push
```

6.2. Cập nhật branch feature bằng rebase

```
git switch feature/payment
git fetch origin
git rebase origin/develop
git push --force-with-lease
```

Lưu ý: Quy tắc an toàn cho người mới: dùng merge trước. Học rebase sau khi đã hiểu conflict, commit history và `force-with-lease`.

Module 7 — Sửa sai: restore, reset, revert, stash, cherry-pick

Git mạnh vì có thể sửa sai, nhưng cũng nguy hiểm nếu dùng sai. Phần này nên học theo tình huống.

7.1. Bảng chọn lệnh theo tình huống

Tình huống	Lệnh nên dùng	Mức độ an toàn
Sửa file nhưng chưa commit, muốn bỏ thay đổi	<code>git restore <file></code>	Tương đối an toàn
Đã add nhầm file vào staging	<code>git reset <file></code>	An toàn
Commits sai message gần nhất	<code>git commit --amend</code>	Cẩn thận nếu đã push
Commits sai nhưng chưa push, muốn quay lại	<code>git reset --soft HEAD~1</code>	An toàn hơn hard
Muốn đảo ngược commit đã push	<code>git revert <commit-id></code>	An toàn cho team
Đang code dở, muốn chuyển branch	<code>git stash -u</code>	An toàn nếu nhớ lấy lại
Muốn lấy một commit từ branch khác	<code>git cherry-pick <commit-id></code>	Cần hiểu conflict

7.2. git restore

```
# Bỏ thay đổi của một file chưa commit
git restore src/main/java/UserService.java

# Bỏ toàn bộ thay đổi chưa commit
git restore .
```

7.3. git reset

```
# Bỏ file khỏi staging nhưng không xóa nội dung sửa
git reset src/main/java/UserService.java

# Quay lại trước 1 commit nhưng giữ thay đổi trong staging
git reset --soft HEAD~1

# Quay lại trước 1 commit và giữ thay đổi trong working directory
git reset --mixed HEAD~1

# Quay lại trước 1 commit và xóa luôn thay đổi
# Cực kỳ cẩn thận
git reset --hard HEAD~1
```

Cảnh báo: `reset --hard` có thể làm mất code chưa lưu ở commit khác. Đừng dùng trên dự án thật nếu chưa chắc chắn.

7.4. git revert

```
# Tạo commit mới để đảo ngược một commit cũ
git revert a1b2c3d

git push
```

Trong team, revert thường an toàn hơn reset vì nó không viết lại lịch sử đã push lên GitHub.

7.5. git stash

```
# Cất cả file tracked và untracked
git stash -u

# Xem danh sách stash
git stash list
```



```
# Lấy stash gần nhất ra và xóa khỏi danh sách stash
git stash pop

# Lấy stash nhưng vẫn giữ trong danh sách
git stash apply

# Xóa stash gần nhất
git stash drop
```

7.6. git cherry-pick

```
# Lấy một commit từ branch khác đưa vào branch hiện tại
git switch feature/order
git cherry-pick a1b2c3d
```

Lưu ý: Cherry-pick hữu ích khi chỉ cần lấy một commit cụ thể, không muốn merge cả branch lớn.

Module 8 — GitHub Issues và Project Board

Branch giải quyết cách code. Issues và Projects giải quyết cách quản lý việc cần làm.

8.1. Issue dùng để làm gì?

- Ghi nhận bug cần sửa.
- Mô tả feature cần làm.
- Lưu task kỹ thuật như setup Docker, thêm CI, refactor.
- Trao đổi yêu cầu trước khi code.
- Liên kết với Pull Request để biết code nào xử lý task nào.

8.2. Mẫu issue cho feature

```
## Mục tiêu
Thêm chức năng đăng nhập cho user.

## Yêu cầu
- User đăng nhập bằng email và password
- Password phải được kiểm tra bằng BCrypt
- Nếu đăng nhập đúng, trả về JWT token
- Nếu sai, trả về lỗi 401

## API mong muốn
POST /api/auth/login

## Acceptance Criteria
- Login thành công với tài khoản hợp lệ
- Login thất bại với password sai
- Login thất bại với email không tồn tại
- Có test cho AuthService
```

8.3. Branch từ issue

```
# Issue #12: Add user login API
git switch develop
git pull origin develop
git switch -c feature/12-user-login
```

8.4. Link PR với issue

Closes #12
 Fixes #20
 Resolves #31

Khi PR được merge, issue tương ứng có thể tự đóng nếu dùng các từ khóa như Closes / Fixes / Resolves.

8.5. Project Board

Backlog -> Todo -> In Progress -> In Review -> Done

Cột	Ý nghĩa
Backlog	Ý tưởng hoặc việc chưa ưu tiên.
Todo	Việc đã chọn để làm.
In Progress	Đang code.
In Review	Đã tạo PR, đang review/test.
Done	Đã merge hoặc hoàn tất.

Module 9 — GitHub Actions và CI/CD

CI/CD giúp tự động kiểm tra code. Thay vì đợi người review chạy tay, GitHub có thể tự build, test, lint mỗi khi có Pull Request.

9.1. Khái niệm cơ bản

Thuật ngữ	Giải thích
Workflow	File YAML định nghĩa automation, thường nằm trong .github/workflows/.
Trigger	Sự kiện kích hoạt workflow: push, pull_request, workflow_dispatch.
Job	Một nhóm bước chạy trên runner.
Step	Một lệnh hoặc action cụ thể.
Runner	Máy ảo chạy workflow, ví dụ ubuntu-latest.
Secret	Biến bí mật như token, password, deploy key.

9.2. Ví dụ CI cho Spring Boot/Maven

```

name: Java CI

on:
  pull_request:
    branches: [develop, main]
  push:
    branches: [develop, main]

jobs:
  build:
    runs-on: ubuntu-latest
    
```

```

steps:
  - name: Checkout source code
    uses: actions/checkout@v4

  - name: Set up Java
    uses: actions/setup-java@v4
    with:
      distribution: temurin
      java-version: '17'

  - name: Run tests
    run: ./mvnw clean test

```

9.3. Ví dụ CI cho Node.js

```

name: Node CI

on:
  pull_request:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: npm

      - run: npm ci
      - run: npm test
      - run: npm run build

```

9.4. CI nên kiểm tra gì?

- Project build được.
- Unit test pass.
- Integration test quan trọng pass.
- Lint/format không lỗi.
- Không có secret bị commit.
- Docker image build được nếu dự án dùng Docker.

Lưu ý: CI không thay thế hoàn toàn code review. CI kiểm tra tự động, còn reviewer kiểm tra logic, thiết kế, nghiệp vụ và maintainability.

Module 10 — Secrets, .gitignore và bảo mật repo

Một lỗi rất phổ biến của người mới là push nhầm file chứa mật khẩu, token hoặc key lên GitHub.

10.1. Không nên commit những gì?


```
# Không commit các thứ như:
.env
application-prod.yml có password thật
private.key
id_rsa
*.pem
AWS_ACCESS_KEY
JWT_SECRET
DATABASE_PASSWORD
node_modules/
target/
build/
logs/
```

10.2. Ví dụ .gitignore cho Java/Spring Boot

```
target/
*.class
*.log
.env
.idea/
*.iml
.DS_Store

# Maven wrapper vẫn nên giữ:
!mvnw
!mvnw.cmd
!.mvn/wrapper/maven-wrapper.jar
```

10.3. Ví dụ .gitignore cho Node.js

```
node_modules/
dist/
build/
.env
.env.local
npm-debug.log*
yarn-debug.log*
.DS_Store
```

10.4. Dùng GitHub Secrets

Thay vì ghi password trong code, hãy đưa vào secret/environment variable rồi để CI/CD đọc khi chạy.

```
# Ví dụ biến môi trường trong workflow
env:
  DATABASE_URL: ${secrets.DATABASE_URL}
  DATABASE_USERNAME: ${secrets.DATABASE_USERNAME}
  DATABASE_PASSWORD: ${secrets.DATABASE_PASSWORD}
```

10.5. Lỡ push secret lên GitHub thì làm gì?

17. Ngay lập tức thu hồi hoặc đổi secret/token/password đó ở hệ thống gốc.
18. Xóa secret khỏi code và thêm vào .gitignore nếu cần.
19. Tạo secret mới và lưu đúng nơi như GitHub Secrets hoặc hệ thống quản lý secret.
20. Nếu repo public hoặc secret nghiêm trọng, cần xử lý lịch sử Git bằng công cụ chuyên dụng và xem như secret đã bị lộ.

21. Không chỉ commit thêm một commit xóa secret rồi nghĩ là an toàn, vì secret vẫn nằm trong lịch sử Git.

Module 11— README, tài liệu dự án và cách làm repo dễ hiểu

Repo tốt không chỉ có code chạy được, mà còn phải giúp người khác hiểu dự án chạy như thế nào.

11.1. README nên có gì?

- Tên dự án và mô tả ngắn.
- Techstack: Java, Spring Boot, MySQL, Redis, Docker, React, v.v.
- Yêu cầu cài đặt: Java version, Node version, Docker.
- Cách chạy local.
- Cách chạy test.
- Cấu trúc thư mục.
- API chính hoặc link Swagger/OpenAPI.
- Cách đóng góp code: branch, commit, PR.
- Thông tin môi trường: .env.example.

11.2. Mẫu README cho e-commerce backend

```
# E-Commerce Backend

## Tech Stack
- Java 17
- Spring Boot
- MySQL
- Redis
- Docker Compose

## Run Local
```bash
cp .env.example .env
docker compose up -d
./mvnw spring-boot:run
```

## Run Tests
```bash
./mvnw test
```

## Main APIs
- POST /api/auth/register
- POST /api/auth/login
- GET /api/products
- POST /api/cart/items
- POST /api/orders

## Branch Workflow
- main: production-ready code
- develop: integration branch
- feature/*: new features
- bugfix/*: normal bug fixes
- hotfix/*: urgent production fixes
```


11.3. .env.example

```

DATABASE_URL=jdbc:mysql://localhost:3306/ecommerce
DATABASE_USERNAME=root
DATABASE_PASSWORD=change_me
JWT_SECRET=change_me
REDIS_HOST=localhost
REDIS_PORT=6379

```

Lưu ý: .env.example được commit để người khác biết cần cấu hình gì. File .env thật thì không commit.

Module 12 — Tag, Release và Semantic Versioning

Khi dự án có phiên bản, bạn cần biết tag và release. Tag đánh dấu một commit là một phiên bản cụ thể.

12.1. Semantic Versioning

MAJOR.MINOR.PATCH

1.0.0
1.0.1
1.1.0
2.0.0

| Phần | Khi nào tăng? | Ví dụ |
|-------|--|----------------|
| PATCH | Sửa bug nhỏ, không đổi API. | 1.0.0 -> 1.0.1 |
| MINOR | Thêm chức năng nhưng vẫn tương thích. | 1.0.1 -> 1.1.0 |
| MAJOR | Thay đổi lớn, có thể phá tương thích cũ. | 1.1.0 -> 2.0.0 |

12.2. Tạo tag

```

git switch main
git pull origin main

git tag v1.0.0
git push origin v1.0.0

# Xem danh sách tag
git tag

```

12.3. Release checklist

- Branch main hoặc release đã ổn định.
- CI pass.
- Test quan trọng pass.
- Changelog đã cập nhật.
- Version đã tăng đúng quy tắc.
- Tạo tag và GitHub Release.
- Deploy lên môi trường tương ứng.
- Theo dõi log/error sau deploy.

12.4. Changelog mẫu

```

##v1.1.0

### Added
- Add VNPaypayment integration
- Add order history API

### Fixed
- Fixcart total calculation when item quantity changes

### Changed
- Refactor ProductService validation logic
    
```

Module 13 — Branch Protection Rules

Branch protection giúp bảo vệ main/develop khỏi việc push trực tiếp hoặc merge code chưa kiểm tra.

13.1. Nên bảo vệ branch nào?

- main: luôn nên bảo vệ.
- develop: nên bảo vệ nếu team nhiều người.
- release/*: nên bảo vệ nếu có quy trình release nghiêm túc.

13.2. Rule gợi ý cho main

| Rule | Lý do |
|-------------------------------------|------------------------------------|
| Require pull request before merging | Không cho push trực tiếp vào main. |
| Require approvals | Bắt buộc có người review. |
| Require status checks to pass | CI phải pass trước khi merge. |
| Require conversation resolution | Comment review phải được xử lý. |
| Restrict force pushes | Tránh phá lịch sử main. |
| Restrict deletions | Tránh xóa nhầm branch quan trọng. |

13.3. Quy tắc team nên thống nhất

- Không push trực tiếp vào main.
- Mọi thay đổi phải qua Pull Request.
- Ít nhất 1 reviewer approve trước khi merge.
- CI phải pass.
- Branch feature merge xong thì xóa.
- Không force push lên branch dùng chung nếu chưa báo team.
- Không commit secret, .env, file build, node_modules, target.

Module 14 — Fork, origin và upstream

Khi đóng góp vào repo người khác, bạn thường không có quyền push trực tiếp. Bạn cần fork repo về tài khoản mình.

14.1. Khái niệm

| Tên | Ý nghĩa |
|--------|--|
| origin | Remote repo mặc định bạn clone. Nếu clone từ fork, origin là fork của bạn. |

| Tên | Ý nghĩa |
|--------------|--|
| upstream | Repo gốc mà bạn muốn đóng góp vào. |
| fork | Bản sao reponằm trong tài khoản GitHub của bạn. |
| pull request | Yêu cầu repo gốc nhận thay đổi từ fork/branch của bạn. |

14.2. Flow đóng góp open-source

```
# Clone fork của bạn
git clone https://github.com/your-name/project.git
cd project

# Thêm repo gốc làm upstream
git remote add upstream https://github.com/original-owner/project.git

# Cập nhật code mới từ repo gốc
git fetch upstream
git switch main
git merge upstream/main

# Tạo branch làm việc
git switch -c fix/readme-typo

# Commit và push lên fork
git add.
git commit -m "docs: fix README typo"
git push -u origin fix/readme-typo
```

Module 15 — GitHub trong dự án lớn: monorepo, multirepo, microservices

Khi làm e-commerce hoặc microservices, cách tổ chức repo ảnh hưởng trực tiếp đến branch, CI/CD và release.

15.1. Monorepo

```
ecommerce/
├── user-service/
├── product-service/
├── order-service/
├── payment-service/
├── frontend/
└── docker-compose.yml
```

- Ưu điểm: dễ thấy toàn hệ thống, dễ chạy local bằng docker compose, dễ refactor liên service.
- Nhược điểm: repo lớn, CI cần tối ưu để không build tất cả service mỗi lần.
- Phù hợp khi team nhỏ/vừa hoặc muốn học kiến trúc tổng thể.

15.2. Multirepo

```
user-service repo
product-service repo
order-service repo
payment-service repo
```



```

frontendrepo
infrarepo
```

- Ưu điểm: mỗi service độc lập, quyền truy cập và release riêng.
- Nhược điểm: quản lý version, local setup và thay đổi xuyên service phức tạp hơn.
- Phù hợp team lớn, service ownership rõ ràng.

15.3. Branch trong microservices

| Tình huống | Gợi ý |
|----------------------------------|--|
| Một repo chứa nhiều service | Branch feature có thể đặt theo service: feature/order-checkout, feature/payment-vnpay. |
| Mỗi service một repo | Mỗi repo có workflow riêng, nhưng cần quản lý version/API contract giữa service. |
| Thay đổi API xuyên nhiều service | Cần issue/epic chung, PR ở nhiều repo, thứ tự merge/deploy rõ ràng. |
| Feature dài ngày | Cần nhắc feature flag để merge sớm nhưng chưa bật tính năng. |

Module 16 — Feature Flag và branch sống ngắn

Branch sống quá lâu dễ conflict và khó merge. Feature flag giúp merge code sớm hơn nhưng chưa bật tính năng cho người dùng.

16.1. Ví dụ feature flag

```

payment.vnpay.enabled=false

//Pseudo-code
if (featureFlags.isVnpayEnabled()) {
  return vnpayPaymentService.createPayment(order);
}
return defaultPaymentService.createPayment(order);
```

16.2. Khi nào dùng feature flag?

- Tính năng lớn làm nhiều ngày/tuần.
- Muốn merge từng phần nhỏ vào develop/main nhưng chưa cho user dùng.
- Muốn bật/tắt tính năng theo môi trường: dev, staging, production.
- Muốn rollout từ từ cho một nhóm user.

Lưu ý: Feature flag không thay thế test. Code tắt flag vẫn cần đảm bảo không phá hệ thống.

Module 17 — Chọn workflow: GitHub Flow, Git Flow hay Trunk-Based?

| Workflow | Branch chính | Phù hợp với |
|-------------|---|---|
| GitHub Flow | main + feature/* | Team nhỏ, deploy liên tục, workflow đơn giản. |
| Git Flow | main + develop + feature/release/hotfix | Dự án release theo version, nhiều môi trường. |

| Workflow | Branch chính | Phù hợp với |
|-------------------------|-----------------------------|---|
| Trunk-Based Development | main/trunk, branch rất ngắn | Team có CI/CD mạnh, test tốt, feature flag tốt. |

17.1. Gợi ý chọn cho người mới

- 22. Học GitHub Flow trước: main + feature branch + PR.
- 23. Khi hiểu PR, conflict và release, học Git Flow.
- 24. Khi có CI/CD tốt và test tốt, tìm hiểu Trunk-Based Development.
- 25. Đừng chọn workflow quá phức tạp nếu team chưa đủ kinh nghiệm.

Module 18 — Sổ tay tình huống lỗi thường gặp

18.1. Lỗi commit nhầm branch

```
# Giả sử bạn commit nhầm vào develop nhưng chưa push
# 1. Lưu commit id
git log --oneline

# 2. Tạo branch mới từ commit hiện tại
git switch -c feature/correct-branch

# 3. Quay develop về trước commit nhầm
git switch develop
git reset --hard HEAD~1

# 4. Tiếp tục làm ở branch đúng
git switch feature/correct-branch
```

18.2. Push bị rejected vì remote có commit mới

```
# Lỗi thường gặp:
# rejected because remote contains work that you do not have locally

# Cách xử lý an toàn:
git pull --rebase origin feature/login
# Sửa conflict nếu có
# Sau đó:
git push
```

18.3. Muốn đổi tên branch

```
# Đổi tên branch local hiện tại
git branch -m feature/new-name

# Push branch mới lên GitHub
git push -u origin feature/new-name

# Xóa branch cũ trên GitHub
git push origin --delete feature/old-name
```

18.4. Muốn lấy lại branch đã xóa local

```
git fetch origin
git switch -c feature/login origin/feature/login
```

18.5. Muốn xem ai sửa dòng này

```
git blame src/main/java/com/example/UserService.java
```

18.6. Muốn xem nội dung commit cụ thể

```
git show a1b2c3d
```

18.7. Muốn xem branch đang lệch nhau thế nào

```
git log --oneline --graph --all --decorate
```

Module 19 — Workflow hằng ngày và bài tập thực hành

19.1. Checklist bắt đầu ngày làm việc

```
git switch develop
git pull origin develop

git switch feature/your-task
git merge origin/develop
# hoặc git rebase origin/develop nếu team dùng rebase

# Chạy app/test trước khi code tiếp
```

19.2. Checklist trước khi push

- git status để xem file thay đổi.
- git diff để kiểm tra mình sửa gì.
- Không có .env, secret, file build, file rác.
- Commit message rõ nghĩa.
- Chạy test hoặc ít nhất chạy app local.
- Push lên đúng branch.

19.3. Checklist trước khi merge PR

- PR mô tả rõ mục tiêu và thay đổi.
- Reviewer đã approve.
- CI pass.
- Không còn conversation chưa xử lý.
- Không có conflict.
- Branch target đúng: develop/main.
- Sau merge, xóa branch feature nếu không cần nữa.

19.4. Bài tập thực hành theo cấp độ

| Cấp độ | Bài tập |
|---------|---|
| Level 1 | Tạo repo, commit README, push lên GitHub. |
| Level 2 | Tạo feature branch, push, mở PR, merge, xóa branch. |

| Cấp độ | Bài tập |
|----------|---|
| Level 3 | Tạo conflict giả giữa 2 branch và tự sửa conflict. |
| Level 4 | Dùng stash khi đang codedở rồi chuyển branch. |
| Level 5 | Tạo issue, tạo branch theo issue, PR ghi Closes #issue. |
| Level 6 | Thêm GitHub Actions chạy test giả. |
| Level 7 | Bật branch protection cho main. |
| Level 8 | Tạo tag v1.0.0 và viết changelog. |
| Level 9 | Tạo repo e-commerce mini với README, .gitignore, PR template. |
| Level 10 | Mô phỏng team 2 người: 2 branch, 2 PR, conflict, review, merge. |

Phụ lục A — Bộ lệnh Git/GitHub hay dùng

| Mục đích | Lệnh |
|-------------------------|--|
| Xem trạng thái | git status |
| Xem thay đổi | git diff |
| Add toàn bộ | git add . |
| Commit | git commit -m "feat: message" |
| Push lần đầu branch mới | git push -u origin feature/name |
| Push các lần sau | git push |
| Fetch remote | git fetch origin |
| Pull branch hiện tại | git pull |
| Tạo branch mới | git switch -c feature/name |
| Chuyển branch | git switch branch-name |
| Xóa branch local | git branch -d branch-name |
| Xóa branch remote | git push origin --delete branch-name |
| Xem log đẹp | git log --oneline --graph --all --decorate |
| Stash codedở | git stash -u |
| Lấy stash ra | git stash pop |
| Hủy merge | git merge --abort |
| Hủy rebase | git rebase --abort |
| Revert commit | git revert <commit-id> |
| Tạo tag | git tag v1.0.0 && git push origin v1.0.0 |

Phụ lục B — Template gợi ý cho repo

B.1. Pull Request template

```
## Summary
Mô tả ngắn PR này làm gì.
```

```
##Changes
- ...
- ...

##How to Test
1....
2....

##Checklist
- [ ] Code chạy local
- [ ] Đã chạy test
- [ ] Không commit secret/file rác
- [ ] Đã cập nhật tài liệu nếu cần

##Related Issue
Closes #...
```

B.2. Issue template cho bug

```
##Mô tả lỗi
Lỗi xảy ra ở đâu?

##Cách tái hiện
1....
2....
3....

##Kết quả hiện tại
...

##Kết quả mong muốn
...

##Môi trường
- OS:
- Browser/App version:
- Backend version:

##Log/Screenshot
...
```

B.3. Quy tắc đặt tên branch

```
feature/12-user-login
bugfix/24-cart-total
hotfix/payment-timeout
docs/update-readme
chore/setup-docker
ci/github-actions
```

Kết luận

Để dùng GitHub tốt trong dự án thật, bạn không chỉ cần biết branch. Bạn cần hiểu toàn bộ flow: task -> branch -> commit -> push -> Pull Request -> review -> CI -> merge -> release -> bảo trì. Khi flow này rõ, dự án lớn sẽ bớt rối, dễ kiểm soát chất lượng và dễ làm việc nhóm hơn.

Gợi ý: Lộ trình học ngắn nhất: Git cơ bản → Branch → Pull Request → Conflict → Review → Issues/Projects → CI/CD → Security → Release → Branch Protection.